Homework Exercises 8

Dr. Sarvar Abdullaev
`s.abdullaev@inha.uz`

November 12, 2022

After completing all lab exercises below, zip all your solution files into a single archive with your student ID (e.g. `u210023.zip`) and upload it to eClass before the official deadline.

1 Designing Algorithms

You should design your algorithms first in pseudocode and then implement them in Python.

1. Check the visualisation of MergeSort in Figure 1. You can even play it interactively here: `https://visualgo.net/en/sorting`. Below is the pseudocode of the `mergeSort` algorithm with an auxiliary

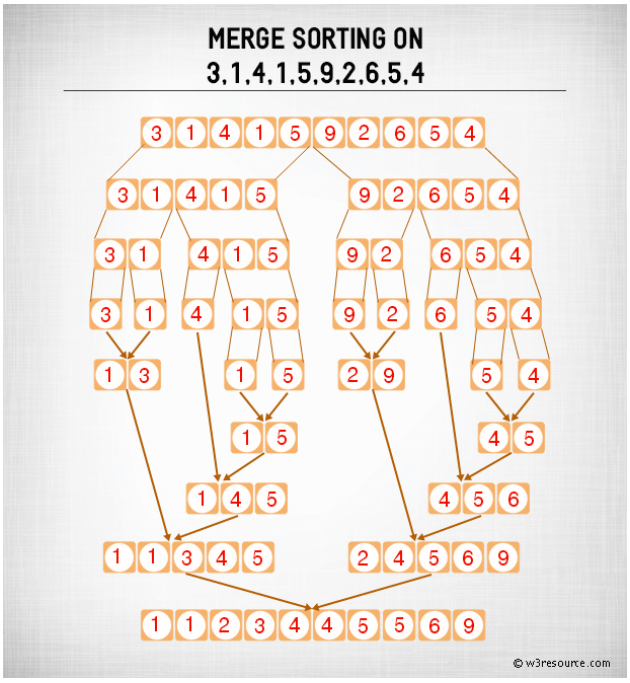

Figure 1: MergeSort example

Python function `merge`. You should implement `mergeSort` in Python and save your program as `mergeSort.py`. It is a recursive algorithm, so determine where are the *divide*, *conquer* and *combine* components of the algorithm.

```
def mergeSort(lst):
    if lst has one element:
        return lst

    left = left half of the lst
    right = right half of the lst

    left = merge_sort(left)
    right = merge_sort(right)
    return merge(left, right)

# combines 2 sorted lists into 1 list.
def merge(left, right):
    result = []
    left_idx, right_idx = 0, 0
    while left_idx < len(left) and right_idx < len(right):
        # change the direction of this comparison to change the direction of the sort
        if left[left_idx] <= right[right_idx]:
            result.append(left[left_idx])
            left_idx += 1
        else:
            result.append(right[right_idx])
            right_idx += 1

    if left_idx < len(left):
        result.extend(left[left_idx:])
    if right_idx < len(right):
        result.extend(right[right_idx:])
    return result
```

2. Design an algorithm using pseudocode for finding all the factors of a positive integer. For example, in the case of the integer 12, your algorithm should report the values 1, 2, 3, 4, 6, and 12. Then implement your algorithm in Python. Save your pseudocode program as `ps1-3.txt` and your Python program as `ps1-3.py`.

3. Check out *Horner's rule* for computing polynomial functions.

$$f(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n = a_0 + x(a_1 + x(a_2 + ... + x(a_{n-1} + a_nx)\dots)) \quad (1)$$

   Implement a recursive Python function which takes as arguments coefficients of the polynomial (e.g $a = [3, 2, 1]$) and the value of x, and returns the result of the computation. Save your program as `ps2-5.py`.

4. Tower of Hanoi is a mathematical puzzle shown in Figure 2 where we have three rods and n disks. The objective of the puzzle is to move the entire stack to another rod. Moving of disks should follow the following rules:

   (a) Only one disk can be moved at a time

   (b) A disk can only be moved if it is the uppermost disk on a stack

   (c) No disk may be placed on top of a smaller disk

   Design a recursive algorithm using pseudocode for solving this mathematical problem. Ask 3 important questions before you try solving it.

   (a) **Divide**: Can I reduce the problem size? If yes, how?

   (b) **Conquer**: What is the simplest problem of this kind? How would I solve it?

   (c) **Combine**: This is the most important and difficult part. How can I use the solution to a smaller problem to obtain a solution to a bigger problem? What would a solution to a smaller problem look like? Imagine the solution to a smaller problem in the context of a bigger problem.

   Then implement your pseudocode in Python. Save your pseudocode program as `ps1-4.txt` and your Python program as `ps1-4.py`.

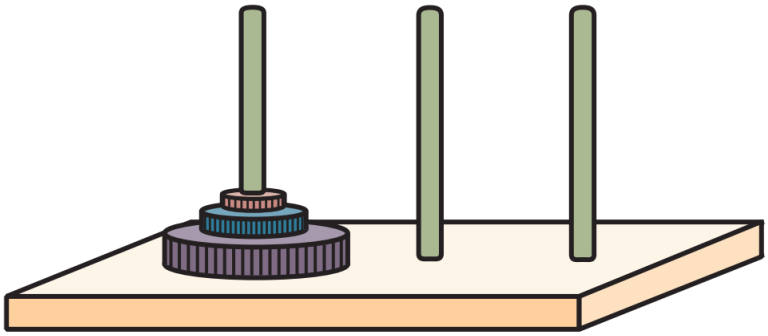

Figure 2: Tower of Hanoi

2 Python Tasks

Watch **Week 6: Algorithmic Complexity** on this course. Then complete following Python tasks.

1. Call the function `CodeWrite` (defined below) with the value 1 and record the values that are printed on your screen. Save your output results in `ps2-1.txt`. What does this function do? Determine *initialize*, *test* and *modify* components of this function.

```
def CodeWrite (num):
    while (num < 100):
        flag = 0
        i = 2
        while(i <= num/2):
            if(num % i == 0):
                flag = 1
                break
            i = i + 1
        if flag == 0:
            print(num)
        num = num + 1
```

2. Call the function `MysteryPrint` (defined below) with the value 2 and record the values that are printed. Save your output results in `ps2-2.txt`

```
def MysteryPrint(N):
    if (N > 0):
        print(N)
        MysteryPrint(N - 2)
    else:
        print(N)
        if (N > -1):
            MysteryPrint(N + 1)
```

3. Write a Python program to find the first duplicate element in a given array of integers. Return -1 If there are no such elements. Save your program as `ps2-3.py`.

4. Write a Python function which takes as an argument a list, and returns a dictionary which gives the frequency of the elements in a list. For example, if the list is `[2,1,2,3,1,9]`, the result should be `{0:0, 1:2, 2:2, 3:1, 4:0, 5:0, 6:0, 7:0, 8:0, 9:1}`. Save your program as `ps2-4.py`.

5. Write a Python program of recursion list sum. For example, if `[1, 2, [3,4], [5,6]]` is given, you should output 21. Save your program as `ps2-5.py`.